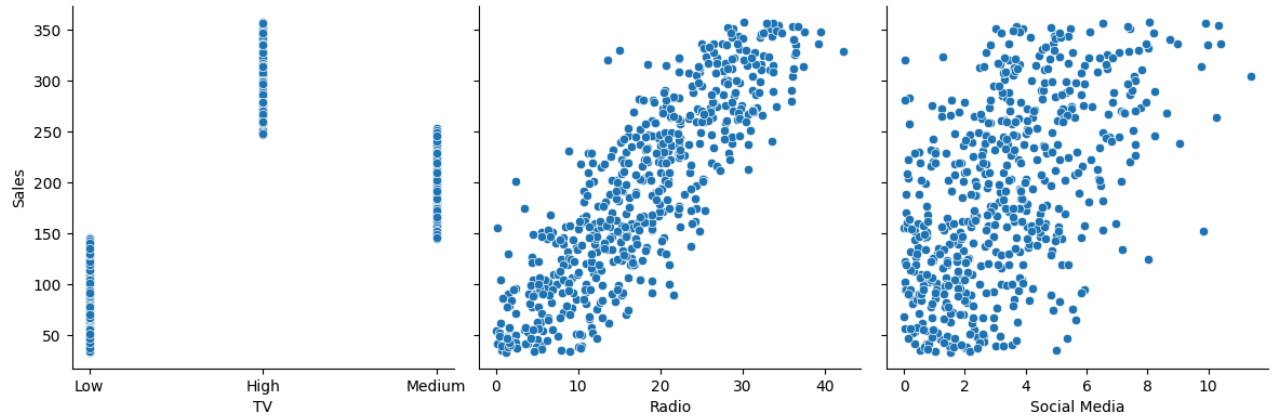
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("marketing_sales_data (2).csv")

# Quick overview
print(df.head())
print(df.describe())

# Pairplot to visualize relationships
sns.pairplot(df, x_vars=['TV','Radio','Social Media'], y_vars='Sales', height=4, aspect=1, kind='scatte
plt.show()
```

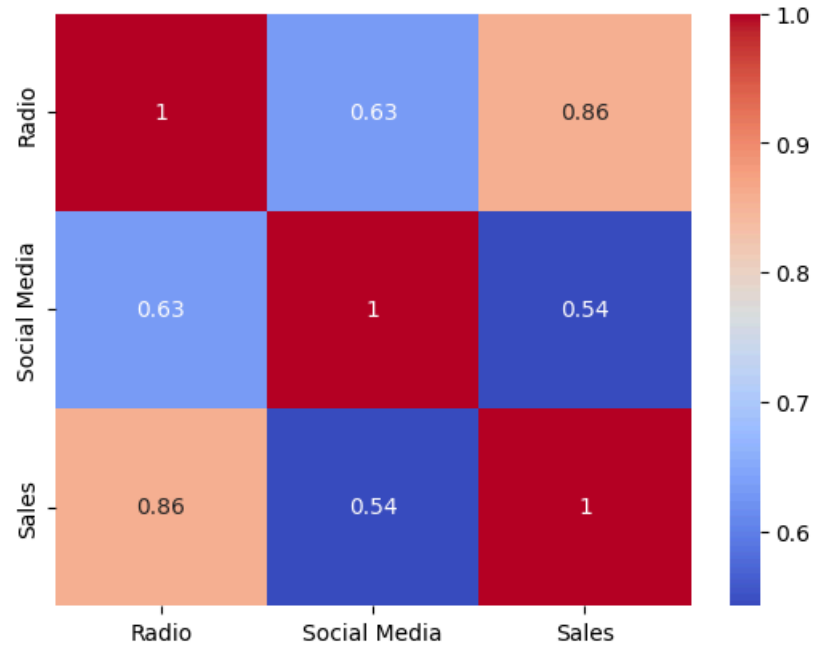
```
       TV      Radio  Social Media Influencer       Sales
0     Low   3.518070      2.293790      Micro   55.261284
1     Low   7.756876      2.572287       Mega   67.574904
2    High  20.348988      1.227180      Micro  272.250108
3  Medium  20.108487      2.728374       Mega  195.102176
4    High  31.653200      7.776978       Nano  273.960377
            Radio  Social Media       Sales
count  572.000000    572.000000  572.000000
mean    17.520616      3.333803  189.296908
std      9.290933      2.238378   89.871581
min      0.109106      0.000031   33.509810
25%     10.699556      1.585549  118.718722
50%     17.149517      3.150111  184.005362
75%     24.606396      4.730408  264.500118
max     42.271579     11.403625  357.788195
```



Check for Multicollinearity

```
# Check for Multicollinearity

from statsmodels.stats.outliers_influence import variance_inflation_factor

# Correlation heatmap
# Exclude 'TV' as it is a categorical column and not directly suitable for numerical correlation

sns.heatmap(df[['Radio','Social Media','Sales']].corr(), annot=True, cmap="coolwarm")
plt.show()

# VIF calculation
# Exclude 'TV' from VIF as it is categorical
X = df[['Radio','Social Media']]
vif_data = pd.DataFrame()
```

```
vif_data["feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
print(vif_data)
# VIF > 10 indicates problematic multicollinearity.
# If detected, consider dropping or combining variables.
```




```
         feature       VIF
0          Radio  5.170922
1  Social Media  5.170922
```

Build Multiple Linear Regression Model

```
import statsmodels.api as sm
import pandas as pd

X = df[['TV','Radio','Social Media']]
y = df['Sales']

# Convert categorical 'TV' column to numerical using one-hot encoding
X = pd.get_dummies(X, columns=['TV'], drop_first=True)

# Ensure all columns in X are numeric (float) before adding constant and fitting the model
X = X.astype(float)

# Add constant for intercept
X = sm.add_constant(X)

model = sm.OLS(y, X).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.904
Model:                            OLS   Adj. R-squared:                  0.903
Method:                 Least Squares   F-statistic:                     1335.
Date:                Sat, 20 Jun 2026   Prob (F-statistic):          7.29e-287
Time:                        18:34:06   Log-Likelihood:                -2713.9
No. Observations:                 572   AIC:                             5438.
Df Residuals:                     567   BIC:                             5460.
Df Model:                           4
```

```
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          218.6473      6.290     34.761      0.000     206.293     231.002
Radio            2.9891      0.234     12.763      0.000       2.529       3.449
Social Media    -0.1500      0.673     -0.223      0.824      -1.471       1.171
TV_Low        -154.3121      4.934    -31.277      0.000    -164.003    -144.622
TV_Medium      -75.3279      3.628    -20.763      0.000     -82.454     -68.202
==============================================================================
Omnibus:                       60.947   Durbin-Watson:                   1.872
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               18.035
Skew:                           0.046   Prob(JB):                     0.000121
Kurtosis:                       2.135   Cond. No.                         144.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Evaluate Model Performance

```
# Define predictors and response using actual column names
X = df[['TV','Radio','Social Media']]
y = df['Sales']

# Convert categorical 'TV' column to numerical using one-hot encoding
X = pd.get_dummies(X, columns=['TV'], drop_first=True)

# Ensure all columns in X are numeric (float)
X = X.astype(float)

# Adds intercept term
X = sm.add_constant(X)

# Fit regression model
model = sm.OLS(y, X).fit()

# Print summary
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.904
Model:                            OLS   Adj. R-squared:                  0.903
Method:                 Least Squares   F-statistic:                     1335.
Date:                Sat, 20 Jun 2026   Prob (F-statistic):          7.29e-287
Time:                        18:34:06   Log-Likelihood:                -2713.9
No. Observations:                 572   AIC:                             5438.
Df Residuals:                     567   BIC:                             5460.
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          218.6473      6.290     34.761      0.000     206.293     231.002
Radio            2.9891      0.234     12.763      0.000       2.529       3.449
Social Media    -0.1500      0.673     -0.223      0.824      -1.471       1.171
TV_Low        -154.3121      4.934    -31.277      0.000    -164.003    -144.622
TV_Medium      -75.3279      3.628    -20.763      0.000     -82.454     -68.202
==============================================================================
Omnibus:                       60.947   Durbin-Watson:                   1.872
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               18.035
Skew:                           0.046   Prob(JB):                     0.000121
Kurtosis:                       2.135   Cond. No.                         144.
```

```
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Interpret Coefficients

```
import statsmodels.api as sm

X = df[['TV', 'Radio', 'Social Media']]
y = df['Sales']

# Convert categorical 'TV' column to numerical using one-hot encoding
X = pd.get_dummies(X, columns=['TV'], drop_first=True)

# Ensure all columns in X are numeric (float)
X = X.astype(float)

# Add constant for intercept
X = sm.add_constant(X)

# Fit regression model
model = sm.OLS(y, X).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.904
Model:                            OLS   Adj. R-squared:                  0.903
Method:                 Least Squares   F-statistic:                     1335.
Date:                Sat, 20 Jun 2026   Prob (F-statistic):          7.29e-287
Time:                        18:34:43   Log-Likelihood:                -2713.9
No. Observations:                 572   AIC:                             5438.
Df Residuals:                     567   BIC:                             5460.
Df Model:                           4
Covariance Type:            nonrobust
================================================================================
                   coef    std err          t      P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------
const          218.6473      6.290     34.761      0.000     206.293     231.002
Radio            2.9891      0.234     12.763      0.000       2.529       3.449
Social Media    -0.1500      0.673     -0.223      0.824      -1.471       1.171
TV_Low        -154.3121      4.934    -31.277      0.000    -164.003    -144.622
TV_Medium      -75.3279      3.628    -20.763      0.000     -82.454     -68.202
==============================================================================
Omnibus:                       60.947   Durbin-Watson:                   1.872
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               18.035
Skew:                           0.046   Prob(JB):                     0.000121
Kurtosis:                       2.135   Cond. No.                         144.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Business Recommendation

```
# Define predictors and target
X = df[['TV', 'Radio', 'Social Media']]
y = df['Sales']

# Convert categorical 'TV' column to numerical using one-hot encoding
X = pd.get_dummies(X, columns=['TV'], drop_first=True)
```

```
# Ensure all columns in X are numeric (float)
X = X.astype(float)

# Add constant for intercept
X = sm.add_constant(X)

# Fit regression model
model = sm.OLS(y, X).fit()

# Print summary
print(model.summary())

# Extract key metrics
print("\nCoefficients:\n", model.params)
print("\nP-values:\n", model.pvalues)
print("\nAdjusted R²:", model.rsquared_adj)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.904
Model:                            OLS   Adj. R-squared:                  0.903
Method:                 Least Squares   F-statistic:                     1335.
Date:                Sat, 20 Jun 2026   Prob (F-statistic):          7.29e-287
Time:                        19:03:33   Log-Likelihood:                -2713.9
No. Observations:                 572   AIC:                             5438.
Df Residuals:                     567   BIC:                             5460.
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          218.6473      6.290     34.761      0.000     206.293     231.002
Radio            2.9891      0.234     12.763      0.000       2.529       3.449
Social Media    -0.1500      0.673     -0.223      0.824      -1.471       1.171
TV_Low        -154.3121      4.934    -31.277      0.000    -164.003    -144.622
TV_Medium      -75.3279      3.628    -20.763      0.000     -82.454     -68.202
==============================================================================
Omnibus:                       60.947   Durbin-Watson:                   1.872
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               18.035
Skew:                           0.046   Prob(JB):                     0.000121
Kurtosis:                       2.135   Cond. No.                         144.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Coefficients:
 const          218.647300
Radio            2.989132
Social Media    -0.149951
TV_Low        -154.312057
TV_Medium      -75.327863
dtype: float64

P-values:
 const          1.192076e-142
Radio           5.661677e-33
Social Media    8.237009e-01
TV_Low         1.518358e-125
TV_Medium       1.207701e-71
dtype: float64

Adjusted R²: 0.9033396396227038
```